Tractament del text abans de les xarxes recurrents

En els capítols anteriors hem treballat amb dades que es poden representar de manera natural com un vector de característiques de mida fixa: una fila d'una taula o els píxels d'una imatge.

El text, en canvi, no encaixa directament en aquest esquema. Una frase és una seqüència de símbols discrets: lletres que es converteixen en paraules, on la seva longitud varia d'un exemple a un altre. L'ordre dels elements és essencial per al significat i, a més, el llenguatge és ambigu: una mateixa paraula pot tenir significats diferents segons el context.

Abans de veure les xarxes neuronals recurrents, que són models dissenyats específicament per processar seqüències, en aquest capítol repassarem com s'ha abordat tradicionalment el tractament automàtic del text: com es prepara, com es converteix en una representació numèrica que un model pugui processar i quines limitacions tenen aquests enfocaments. Aquestes limitacions seran la motivació que ens durà a treballar amb les xarxes recurrents.

El repte del text

Si seguim la notació que vam definir al primer capítol, sabem que cada mostra del conjunt de dades és un vector $\mathbf{x}^{(i)} = (x_1^{(i)}, \dots, x_n^{(i)})$, amb $x_j^{(i)} \in \mathbb{R}$, a més, també sabem que tots els models que hem vist esperen que n sigui el mateix per a totes les mostres. Amb el text ens trobem amb diversos problemes:

- **Les dades són discretes.** Les paraules són símbols, no nombres. No té sentit dir que la paraula "moix" és més gran que "ca", ni sumar-les. Cal decidir com les convertim en valors numèrics.
- **La longitud és variable.** Un tuit pot tenir 5 paraules i un article de premsa, 2.000. En canvi, la capa d'entrada d'un Perceptró multicapa té un nombre fix d'unitats.
- **L'ordre és important.** "El ca mossega l'elefant" i "l'elefant mossega el ca" contenen exactament les mateixes paraules, però no signifiquen el mateix.
- **El context determina el significat.** La paraula "banc" no vol dir el mateix a "m'assec en un banc del parc" que a "he obert un compte al banc".
- **El vocabulari és molt gran i obert.** Una llengua té desenes de milers de paraules, apareixen paraules noves constantment, i moltes paraules apareixen molt poques vegades en un text.

El conjunt de tècniques que permeten als ordinadors processar el llenguatge humà s'anomena **processament del llenguatge natural** (*Natural Language Processing*, *NLP*). A continuació veurem les etapes clàssiques d'aquest processament: la preparació del text, la seva representació numèrica i els models que

s'hi apliquen.

Preprocessament del text

Abans de representar numèricament un text, és habitual aplicar-li una sèrie de transformacions per reduir-ne la variabilitat innecessària. En conjunt, aquestes transformacions s'anomenen preprocessament.

Tokenització

La tokenització consisteix a dividir el text en unitats mínimes anomenades tokens, que normalment corresponen a paraules i signes de puntuació. Tot i que pot semblar trivial (n'hi hauria prou de separar per espais), cada llengua té les seves particularitats. En català, per exemple, cal decidir com tractar:

- Els apòstrofs: "l'home" s'hauria de separar en "l'" i "home".
- Les contraccions: "al" és en realitat "a" + "el".
- El punt volat: "tranquil · lament" és una sola paraula, encara que contingui un signe de puntuació.
- Les paraules compostes amb guionet: "porta-retrats", o els pronoms febles: "dona-m'ho".

Així, la frase "Els moixos dormien profundament al sofà." es podria tokenitzar com:

$$[\text{Els}, \text{moixos}, \text{dormien}, \text{profundament}, \text{al}, \text{sofà}, .]$$

Normalització

Per evitar que el model consideri diferents paraules que en realitat són la mateixa, s'apliquen diverses normalitzacions:

- **Conversió a minúscules**: "Els" i "els" passen a ser el mateix token.
- **Eliminació de la puntuació** i, en alguns casos, dels accents o de caràcters especials.
- **Eliminació de paraules buides**: també conegudes com a *stopwords*: articles, preposicions, conjuncions, etc. ("el", "la", "de", "i", "que"…), que són molt freqüents però aporten poca informació sobre el contingut del text.

Stemming i lematització

Una mateixa paraula pot aparèixer amb moltes formes diferents: "moix", "moixos", "moixa", "moixes"; o "dormir", "dormo", "dormien", "dormirem". Per agrupar-les hi ha dues estratègies:

- ***Stemming* (extracció de l'arrel):** s'eliminen els sufixos de la paraula aplicant regles senzilles. És ràpid, però el resultat no sempre és una paraula real: "dormien" → "dorm", "profundament" → "profund".
- **Lematització:** es substitueix cada paraula pel seu lema, és a dir, la forma que trobaríem al diccionari: "dormien" → "dormir", "moixos" → "moix". És més precisa, però requereix un diccionari i, sovint, una anàlisi gramatical de la frase.

Aplicant tot el procés a la frase anterior (minúscules, sense puntuació, sense paraules buides i lematitzada), obtindríem:

$$[\text{moix},\ \text{dormir},\ \text{profundament},\ \text{sofà}]$$

Cal tenir en compte que cada pas de preprocessament elimina informació. Per a una tasca de classificació temàtica, eliminar "no" com a paraula buida pot ser irrellevant, però en una tasca d'anàlisi de sentiment, "m'agrada" i "no m'agrada" deixarien de distingir-se. El preprocessament adequat depèn sempre de la tasca.

Vocabulari

Un cop preprocessats tots els textos del conjunt de dades (en aquest context s'anomena **corpus**), construïm el **vocabulari** V: el conjunt de tots els tokens diferents que hi apareixen. Cada token rep un índex enter, de 1 a $|V|$. És habitual limitar el vocabulari a les paraules més freqüents (per exemple, les 20.000 més habituals) i substituir la resta per un token especial de paraula desconeguda, com ara `<UNK>`. Aquest problema de les paraules fora del vocabulari (*out-of-vocabulary*, *OOV*) és un dels motius pels quals els models moderns treballen amb fragments de paraules (*subword tokenization*) en lloc de paraules senceres, tal com veurem més endavant en el curs.

Representacions dispersades

Un cop tenim els textos convertits en seqüències de tokens i cada token té un índex dins del vocabulari. Ara cal convertir aquestes seqüències en vectors numèrics.

Codificació *one-hot*

La manera més senzilla de representar una paraula és la codificació ***one-hot***: cada paraula w del vocabulari es representa amb un vector $\mathbf{o}_w \in \{0,1\}^{|V|}$ ple de zeros, excepte a la posició corresponent al seu índex, on hi ha un 1.

Per exemple, amb un vocabulari $V = \{\text{moix},\ \text{ca},\ \text{dorm},\ \text{sofà}\}$:

$$\mathbf{o}_{\text{moix}} = (1,0,0,0), \quad \mathbf{o}_{\text{ca}} = (0,1,0,0), \quad \mathbf{o}_{\text{dorm}} = (0,0,1,0), \quad \mathbf{o}_{\text{sofà}} = (0,0,0,1).$$

Aquesta representació té dos inconvenients importants:

1. **Dimensionalitat:** si el vocabulari té 50.000 paraules, cada paraula és un vector de 50.000 components, dels quals només un és diferent de zero. Per això es diu que és una representació **dispersa** (*sparse*).
2. **No captura cap relació entre paraules:** tots els vectors *one-hot* són ortogonals entre si, és a dir, $\mathbf{o}_{\text{moix}}^{\top}\mathbf{o}_{\text{ca}} = 0$. Per al model, "moix" és tan diferent de "ca" com de "sofà".

A més, la codificació *one-hot* representa paraules, no textos. Per representar un text sencer, podríem concatenar els vectors de cada paraula, però aleshores la mida del vector dependria de la longitud del text, cosa que ens torna al problema de la longitud variable.

Bossa de paraules (*bag of words*)

La solució clàssica per obtenir un vector de mida fixa per a un text de qualsevol longitud és la tècnica **bossa de paraules** (*bag of words*, *BoW*). La idea és representar cada document com un vector $\mathbf{x} \in \mathbb{R}^{|V|}$ on cada component x_j és el nombre de vegades que la paraula j del vocabulari apareix al document. Equivalentment, és la suma dels vectors *one-hot* de totes les paraules del document.

Considerem el següent corpus de tres documents (sense eliminar paraules buides, per simplicitat):

- d_1: "el moix dorm al sofà"
- d_2: "el ca dorm al jardí"
- d_3: "el moix i el ca juguen"

El vocabulari, ordenat alfabèticament, és $V = \{\text{al, ca, dorm, el, i, jardí, juguen, moix, sofà}\}$, amb $|V| = 9$. La representació de cada document és:

	al	ca	dorm	el	i	jardí	juguen	moix	sofà
$\mathbf{x}^{(1)}$	1	0	1	1	0	0	0	1	1
$\mathbf{x}^{(2)}$	1	1	1	1	0	1	0	0	0
$\mathbf{x}^{(3)}$	0	1	0	2	1	0	1	1	0

Ara cada document és un vector de la mateixa dimensió, independentment de la seva longitud, i per tant ja el podem utilitzar com a entrada de qualsevol dels models que hem estudiat.

El preu que paguem és que **es perd completament l'ordre de les paraules**: el document es tracta com una "bossa" on les paraules estan barrejades. Les frases "el ca mossega l'home" i "l'home mossega el ca" tindrien exactament el mateix vector.

TF-IDF

La bossa de paraules dona el mateix pes a totes les paraules. Tanmateix, a l'exemple anterior la paraula "el" apareix a tots els documents i, per tant, no ens ajuda gens a distingir-los, mentre que "sofà" només apareix a d_1 i el caracteritza molt bé. La ponderació **TF-IDF** (*term frequency – inverse document frequency*) corregeix aquest fet combinant dos factors:

- La **freqüència del terme** $\mathrm{tf}(t, d)$: el nombre de vegades que el terme t apareix al document d (el mateix valor que a la bossa de paraules).
- La **freqüència inversa de document** $\mathrm{idf}(t)$: penalitza els termes que apareixen a molts documents del corpus.

$$\mathrm{idf}(t) = \log \frac{N}{\mathrm{df}(t)},$$

on N és el nombre total de documents del corpus i $\mathrm{df}(t)$ és el nombre de documents que contenen el terme t. El pes final de cada terme en cada document és:

$$\text{tf-idf}(t, d) = \mathrm{tf}(t, d) \cdot \mathrm{idf}(t).$$

Aplicant-ho al nostre exemple ($N = 3$, logaritme natural):

Terme	$\mathrm{df}(t)$	$\mathrm{idf}(t)$
el	3	$\log(3/3) = 0$
ca, moix, dorm, al	2	$\log(3/2) \approx 0.405$
sofà, jardí, i, juguen	1	$\log(3/1) \approx 1.099$

La paraula "el", que apareix a tots els documents, rep un pes 0 i desapareix de la representació, mentre que les paraules exclusives d'un document reben el pes més alt. Així, el vector TF-IDF de d_1 seria:

$$\mathbf{x}^{(1)} \approx (0.405,\ 0,\ 0.405,\ 0,\ 0,\ 0,\ 0,\ 0.405,\ 1.099).$$

Existeixen moltes variants d'aquesta fórmula (freqüències normalitzades per la longitud del document, suavitzat de l'idf, normalització del vector final, etc.), però la idea és sempre la mateixa: **una paraula és important per a un document si hi apareix sovint i, alhora, apareix en pocs documents del corpus**.

n-grames

Una manera parcial de recuperar informació sobre l'ordre és utilitzar com a unitats de vocabulari no només paraules individuals, sinó també seqüències de n paraules consecutives, anomenades **n-grames**. Per a la frase "el moix dorm al sofà":

- **Unigrames** ($n = 1$): "el", "moix", "dorm", "al", "sofà".
- **Bigrames** ($n = 2$): "el moix", "moix dorm", "dorm al", "al sofà".
- **Trigrames** ($n = 3$): "el moix dorm", "moix dorm al", "dorm al sofà".

Amb bigrames, "no m'agrada" esdevé un element del vocabulari diferent de "m'agrada", i "el ca mossega" és diferent de "l'home mossega". El problema és que el vocabulari creix molt ràpidament: amb $|V|$ paraules hi ha fins a $|V|^2$ bigrames possibles i $|V|^3$ trigrames, la majoria dels quals no apareixeran mai, o només un cop, al *corpus*. A més, l'ordre només es captura dins d'una finestra local de n paraules.

La dimensionalitat del text

En el primer capítol vam veure que una imatge en color de 400×400 píxels suposa 480.000 característiques per mostra. Amb el text ens trobem en una situació semblant: amb una bossa de paraules, la dimensió de cada mostra és la mida del vocabulari, que habitualment és de l'ordre de 10^4 a 10^5, i encara més si hi afegim n-grames. La diferència és que, mentre que en una imatge tots els píxels tenen un valor, en un vector de text gairebé tots els components són zero: un document de 200 paraules en un vocabulari de 50.000 en té, com a molt, 200 de diferents de zero.

Models clàssics sobre aquestes representacions

Models lineals

Un cop tenim cada document representat com un vector $\mathbf{x} \in \mathbb{R}^{|V|}$, ja podem aplicar directament els models del capítol de models lineals. Per exemple, per classificar correus electrònics com a *spam* ($y = 1$) o no *spam* ($y = 0$), podem fer servir la regressió logística:

$$\hat{y} = \sigma\left(\sum_{j=0}^{|V|} w_j \cdot x_j\right),$$

entrenada amb l'entropia creuada i el descens de gradient exactament com ja coneixem. Cada pes w_j està associat a una paraula del vocabulari, i això fa que aquest model sigui molt interpretable: les paraules amb pesos positius grans ("gratis", "premi", "urgent") empenyen la predicció cap a *spam*, mentre que les que tenen pesos negatius l'empenyen cap a no *spam*.

Un altre model clàssic molt utilitzat per a la classificació de text és Naive Bayes, que estima la probabilitat de cada classe a partir de les freqüències de les paraules, assumint que les paraules són independents entre si donada la classe. Aquests models, combinats amb TF-IDF, van ser l'estat de l'art en classificació de text durant molts anys i encara avui són una línia base útil en moltes aplicacions.

Models de llenguatge n-grama

Un model de llenguatge assigna una probabilitat a una seqüència de paraules $w_1, w_2, \dots, w_T$. Aplicant la regla de la cadena de la probabilitat, podem descompondre aquesta probabilitat com:

$$P(w_1, w_2, \dots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \dots, w_{t-1}).$$

És a dir, modelar el llenguatge equival a saber predir la paraula següent donades totes les anteriors. Aquesta és exactament la tasca que fan els teclats predictius dels mòbils i, com veurem al llarg del curs, és també la base dels grans models de llenguatge actuals.

El problema és que, a mesura que t creix, la història $w_1, \dots, w_{t-1}$ és cada cop més llarga i és pràcticament impossible haver vist exactament aquesta seqüència al *corpus* d'entrenament. Els models n-grama simplifiquen el problema seguint la hipòtesi de Markov: suposen que cada paraula només depèn de les $n-1$ paraules anteriors.

$$P(w_t \mid w_1, \dots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \dots, w_{t-1}).$$

En el cas d'un model de bigrames ($n = 2$), cada paraula només depèn de l'anterior, i aquesta probabilitat s'estima simplement comptant al *corpus*:

$$P(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t)}{C(w_{t-1})},$$

on $C(w_{t-1}, w_t)$ és el nombre de vegades que apareix el bigrama i $C(w_{t-1})$ el nombre de vegades que apareix la paraula w_{t-1}. Amb el *corpus* de l'exemple anterior, la paraula "el" apareix 4 vegades, seguida 2 vegades de "moix" i 2 de "ca":

$$P(\text{moix} \mid \text{el}) = \frac{2}{4} = 0.5, \qquad P(\text{ca} \mid \text{el}) = \frac{2}{4} = 0.5, \qquad P(\text{sofà} \mid \text{el}) = \frac{0}{4} = 0.$$

L'últim cas mostra el problema principal d'aquests models: la dispersió de les dades. Qualsevol seqüència que no aparegui al *corpus* rep probabilitat zero, i

per tant tota la frase que la conté també. Per evitar-ho s'utilitzen tècniques de suavitzat (*smoothing*), com el suavitzat de Laplace, que suma 1 a tots els comptatges:

$$P(w_t \mid w_{t-1}) = \frac{C(w_{t-1}, w_t) + 1}{C(w_{t-1}) + |V|}.$$

Augmentar n permet capturar més context, però multiplica el nombre de combinacions possibles i agreuja el problema de la dispersió. A la pràctica, rarament s'utilitzaven models amb un context de més de 5 paraules.

Representacions denses: *word embeddings*

Totes les representacions vistes fins ara comparteixen el mateix problema de fons: les paraules són símbols independents i el model no té cap manera de saber que "gat" i "moix" volen dir el mateix, o que "ca" i "moix" s'assemblen més entre si que "moix" i "sofà". Tot el que el model aprengui sobre "moix" no li serveix de res per a "gat".

La hipòtesi distribucional

La solució parteix d'una idea de la lingüística dels anys cinquanta, la hipòtesi distribucional (Harris, 1954; Firth, 1957): les paraules que apareixen en contextos semblants tendeixen a tenir significats semblants. Si al *corpus* trobem frases com "el _____ dorm al sofà" o "he donat menjar al _____", tant"moix" com "ca" com "gat" encaixen en aquests buits, i per tant podem deduir que són paraules relacionades.

Vectors densos

A partir d'aquesta idea, en lloc de representar cada paraula amb un vector *one-hot* de dimensió $|V|$, la representem amb un vector **dens** $\mathbf{e}_w \in \mathbb{R}^d$ de dimensió molt més petita (habitualment d té una longitud entre 50 i 300), on tots els components tenen valors reals. Aquest vector s'anomena *embedding* de la paraula. L'objectiu és que paraules amb significats semblants tinguin vectors propers en aquest espai.

Tots els *embeddings* es guarden en una matriu $\mathbf{E} \in \mathbb{R}^{|V| \times d}$, on cada fila correspon a una paraula del vocabulari. Obtenir l'*embedding* d'una paraula és equivalent a multiplicar el seu vector *one-hot* per aquesta matriu, cosa que a la pràctica es fa simplement seleccionant la fila corresponent:

$$\mathbf{e}_w = \mathbf{E}^\top \mathbf{o}_w.$$

Per mesurar la semblança entre dues paraules, s'utilitza habitualment la **similitud cosinus**, que mesura el cosinus de l'angle entre els dos vectors:

$$\text{sim}(\mathbf{e}_a, \mathbf{e}_b) = \frac{\mathbf{e}_a^\top \mathbf{e}_b}{\|\mathbf{e}_a\| \, \|\mathbf{e}_b\|}.$$

Aquest valor està entre -1 i 1, i com més proper a 1, més semblants són les paraules. Fixeu-vos que, amb vectors *one-hot*, la similitud cosinus entre dues paraules diferents és sempre 0.

Word2Vec

Els valors de la matriu $\mathbf{E}$ s'aprenen a partir de les dades, com qualsevol altre paràmetre d'un model. El mètode més conegut és **Word2Vec** (Mikolov et al., 2013), que planteja una tasca artificial de predicció a partir d'un corpus molt gran de text sense etiquetar.

En la variant *skip-gram*, el model rep una paraula central w_t i ha de predir les paraules que l'envolten dins d'una finestra de mida c. Per exemple, a la frase "el moix dorm al sofà", amb $c = 2$ i paraula central "dorm", el model hauria de predir "el", "moix", "al" i "sofà". L'objectiu és maximitzar la probabilitat del context per a totes les posicions del corpus:

$$\frac{1}{T} \sum_{t=1}^{T} \sum_{\substack{-c \leq j \leq c \\ j \neq 0}} \log P(w_{t+j} \mid w_t),$$

on la probabilitat es calcula amb la funció *softmax* que ja vam veure en el capítol de xarxes neuronals:

$$P(o \mid c) = \frac{e^{\mathbf{u}_o^\top \mathbf{v}_c}}{\sum_{w=1}^{|V|} e^{\mathbf{u}_w^\top \mathbf{v}_c}},$$

amb $\mathbf{v}_c$ el vector de la paraula central i $\mathbf{u}_o$ el vector de la paraula de context. És a dir, es tracta d'una xarxa neuronal molt senzilla, amb una única capa oculta lineal, entrenada amb el descens de gradient. La tasca de predicció en si no ens interessa: el que ens interessa són els pesos que la xarxa ha après, que són precisament els *embeddings*. Com que dues paraules que apareixen en contextos semblants han de predir el mateix context, l'entrenament les acaba col · locant a prop una de l'altra.

Altres mètodes populars per obtenir *embeddings* són **GloVe** (Pennington et al., 2014), basat en les estadístiques globals de coocurrència de les paraules, i **FastText** (Bojanowski et al., 2017), que també té en compte els fragments interns de les paraules i, per tant, pot generar vectors per a paraules que no ha vist mai.

Propietats dels *embeddings*

Els *embeddings* apresos d'aquesta manera presenten certes propietats. A més d'agrupar paraules semblants (els noms d'animals queden a prop entre si, igual que els noms de ciutats o els verbs de moviment), certes relacions semàntiques es corresponen amb direccions concretes de l'espai vectorial. L'exemple més conegut és:

$$\mathbf{e}_{\text{rei}} - \mathbf{e}_{\text{home}} + \mathbf{e}_{\text{dona}} \approx \mathbf{e}_{\text{reina}}.$$

La diferència entre "rei" i "home" codifica, aproximadament, la noció de "reialesa", i sumant-la a "dona" s'obté un vector molt proper a "reina". De manera semblant, $\mathbf{e}_{\text{París}} - \mathbf{e}_{\text{França}} + \mathbf{e}_{\text{Itàlia}} \approx \mathbf{e}_{\text{Roma}}$.

Cal tenir en compte, però, que els *embeddings* també aprenen els biaixos presents en els textos d'entrenament (per exemple, associacions estereotipades entre professions i gènere), un aspecte que cal considerar quan s'utilitzen en aplicacions reals.

Un altre avantatge pràctic és que els *embeddings* es poden entrenar una sola vegada amb un corpus enorme (per exemple, tota la Viquipèdia) i després reutilitzar-se com a representació d'entrada en altres tasques amb molt menys dades etiquetades. Aquesta idea de **preentrenar** una representació i reutilitzar-la és una de les claus de l'aprenentatge profund aplicat al llenguatge.

Limitacions

Amb els *embeddings* es resol el problema de representar cada paraula de manera significativa. Però continuem tenint el problema de representar un text sencer, que és una seqüència de longitud variable de vectors $\mathbf{e}_{w_1}, \mathbf{e}_{w_2}, \ldots, \mathbf{e}_{w_T}$. Amb les eines que tenim fins ara, hi ha dues opcions:

1. **Combinar tots els vectors en un de sol:** per exemple, fent-ne la mitjana:
$$\mathbf{x} = \frac{1}{T} \sum_{t=1}^{T} \mathbf{e}_{w_t}.$$
Obtenim un vector de mida fixa d, però tornem a perdre completament l'ordre de les paraules, igual que amb la bossa de paraules.

2. **Concatenar els vectors d'una finestra fixa de paraules:** És l'enfocament del model de llenguatge neuronal de Bengio et al. (2003): per predir la paraula w_t, es concatenen els *embeddings* de les $n-1$ paraules anteriors i s'introdueixen en un Perceptró multicapa:
$$\mathbf{x} = [\mathbf{e}_{w_{t-n+1}};\ \ldots;\ \mathbf{e}_{w_{t-1}}] \in \mathbb{R}^{(n-1)d}.$$

Aquest model ja no pateix el problema de la dispersió dels n-grames, perquè paraules semblants tenen vectors semblants. Però el context continua limitat a una finestra fixa de $n-1$ paraules, i a més cada posició de la finestra té els seus propis pesos: el que el model aprèn sobre una paraula a la posició 1 no li serveix per a la mateixa paraula a la posició 3.

Si resumim les limitacions de tots els enfocaments que hem vist, necessitaríem un model que pugui processar seqüències de longitud arbitrària, sense truncarles ni haver de fixar una finestra. A més, tingui en compte l'ordre dels elements de la seqüència. Comparteixi els paràmetres entre totes les posicions, de manera que el que aprèn sobre una paraula li serveixi independentment d'on aparegui. Finalment, també ha de poder recordar informació de parts anteriors de la seqüència, potencialment llunyanes.

La idea de les xarxes neuronals recurrents és llegir la seqüència element a element, com fem les persones quan llegim, i mantenir un estat intern $\mathbf{h}_t$ que resumeix tot el que s'ha llegit fins aquell moment.

Exemple pràctic

Les representacions de bossa de paraules i TF-IDF estan implementades a la llibreria `scikit-learn`:

```
from sklearn.feature_extraction.text import CountVectorizer, TfidfVectorizer

corpus = [
    "el moix dorm al sofà",
    "el ca dorm al jardí",
    "el moix i el ca juguen",
]

# Bossa de paraules
bow = CountVectorizer(token_pattern=r"(?u)\b\w+\b")  # accepta paraules d'una lletra, com "
X_bow = bow.fit_transform(corpus)
print(bow.get_feature_names_out())
print(X_bow.toarray())

# TF-IDF
tfidf = TfidfVectorizer(token_pattern=r"(?u)\b\w+\b")
X_tfidf = tfidf.fit_transform(corpus)
print(X_tfidf.toarray().round(3))

# Bigrames
bigrames = CountVectorizer(ngram_range=(2, 2))
bigrames.fit(corpus)
print(bigrames.get_feature_names_out())
```

Hem de tenir en compte que, per defecte, `TfidfVectorizer` utilitza una versió

suavitzada de l'idf, $\text{idf}(t) = \log \frac{1+N}{1+\text{df}(t)} + 1$, i normalitza cada vector perquè tingui norma 1. Per aquest motiu, els valors obtinguts no coincideixen exactament amb els calculats a mà a l'apartat anterior, tot i que la idea és la mateixa.

Conclusions

En aquest capítol hem vist que el text presenta reptes propis que el diferencien de les dades tabulars o de les imatges: és discret, de longitud variable, depèn de l'ordre i del context, i té un vocabulari molt gran. Hem repassat les etapes clàssiques del seu tractament: el preprocessament (tokenització, normalització, *stemming* i lematització), les representacions dispersades (*one-hot*, bossa de paraules, TF-IDF i n-grames) i els models clàssics que s'hi apliquen, com la regressió logística per a la classificació de text o els models de llenguatge n-grama. Finalment, hem vist com els *word embeddings* permeten representar les paraules amb vectors densos que capturen relacions de significat.

Totes aquestes tècniques, però, comparteixen una mateixa limitació: o bé perden l'ordre de les paraules, o bé només en consideren una finestra fixa. Les xarxes neuronals recurrents, que veurem a continuació, neixen precisament per processar seqüències completes de longitud arbitrària mantenint una memòria del que s'ha vist anteriorment.

Referències

- Bengio, Y., Ducharme, R., Vincent, P. i Jauvin, C. (2003). A Neural Probabilistic Language Model. *Journal of Machine Learning Research*, 3, 1137–1155.
- Bojanowski, P., Grave, E., Joulin, A. i Mikolov, T. (2017). Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics*, 5, 135–146.
- Firth, J. R. (1957). A synopsis of linguistic theory 1930–1955. *Studies in Linguistic Analysis*, 1–32.
- Harris, Z. S. (1954). Distributional Structure. *Word*, 10(2–3), 146–162.
- Mikolov, T., Chen, K., Corrado, G. i Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. *arXiv:1301.3781*.
- Pennington, J., Socher, R. i Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. *Proceedings of EMNLP*, 1532–1543.